

Bad Work

Work without a control loop and a state structure isn't worth doing

Alexander Vityaz

Corezoid Inc., Dnipro, Ukraine

alexander.vityaz@gmail.com

corezoid.com · simulator.company

ORCID: [0009-0006-0489-7881](https://orcid.org/0009-0006-0489-7881)

DOI: [10.5281/zenodo.23015137](https://doi.org/10.5281/zenodo.23015137)

[Discuss the paper with Gemini Notebook](#)

Published: 28 September 2026

Abstract

Work that crosses programs, APIs, organizations, and people can outlive any one execution context. A timeout may leave the outcome of an external action unknown; a restart may erase the information needed to decide what is admissible next. This article proposes a compact organizational analogue of Alan Perlis's pairing of a loop and a structured variable: the *control loop* and the *state structure*. The control loop relates observed consequences to a goal and selects clarification, retry, compensation, or escalation. The state structure preserves participants, obligations, confirmed facts, unknown outcomes, and transition history. The proposal is connected to the Conant–Ashby regulator theorem and developed through actor graphs with stateful mediator actors, persistent interaction history, and accounting consequences. A bank debit example separates safety, liveness, and conformance. The practical criterion is a handoff test: another executor should be able to resume unfinished work from saved state without reconstructing the system's memory by hand.

Keywords: control loop; state structure; actor graph; distributed systems; durable execution; verification; organizational memory.

“Why did the Roman Empire collapse? What is the Latin for office automation?”

—Alan Perlis, Epigram 111 [1]

1. Perlis’s Epigram

In 1982, Alan Perlis wrote in Epigram 18: “A program without a loop and a structured variable isn’t worth writing” [1].

What interests me in this line is the pairing: a repeated action and the organized data it operates on. The loop sets the order of repetition; the data structure makes it possible to work with complex content. Together they give a compact description of a large number of computational steps.

In Epigram 29, Perlis considered the control graph and the addition of an edge that creates a cycle; in Epigram 68, he tied data structures to independent, simultaneous processing. For me, these are two reference points: close the loop of control over the shared work, and preserve the independence of its participants.

I propose carrying the pairing of action and data over to the organization of activity: from the loop and the variable to the control loop and the state structure. Here, repetition is given a task: check the consequences of an action against the goal and choose the next step. Data becomes the memory of unfinished work, its obligations, and its confirmed results.

Perlis’s epigrams are available in the Yale University transcription and in an archival edition that includes the meta-epigrams [2, 3].

2. When an Action Extends Beyond a Program

When an action passes through several programs, APIs, and people, its result depends on how the participants interact. One program can finish its work while the overall process is still running. Sending a command, receiving it, and achieving the intended result become separate events.

Two questions arise at this boundary:

- **What happened?** A timeout reports that no response arrived in time. The external operation may have completed, may still be running, or may never have started.
- **What survived?** After a restart, the context has to be recovered: which action was requested, what has already been confirmed, and which obligations remain open.

Suppose a bank has processed a debit, but the response was lost. The model must include an `outcome-unknown` state and a rule for subsequent reconciliation. Retrying the operation requires protection against double execution; compensation requires grounds for carrying it out. The saved state makes it possible to resume work from the point where reliable information ended.

This is exactly what occupies me in my work on Corezoid: how to keep a process controllable when an individual program has stopped but the obligation remains. That is why the whole process of interaction has to be designed, including waiting, uncertainty, and recovery after a failure.

3. The Control Loop and the State Structure

By work I mean here an activity for whose result someone is accountable. The loop can close on a person: they remember the goal, observe the result, and correct the actions. But if the goal stays in the manager’s head and the state of the work stays in message threads, control depends on that manager’s memory and presence. I propose making the state and the rules of control explicit, so that the work can be resumed after a failure or handed off to another executor.

1. **The control loop** links the goal to the consequences of an action. It receives information about what is happening, relates it to the goal and the model, selects an admissible intervention, and checks the result. If the result is unknown or diverges from what was expected, the loop triggers clarification, retry, compensation, or escalation under predefined rules. An acknowledgment confirms a particular stage of the interaction, such as receipt of a command. Whether the goal has been achieved is checked separately.
2. **The state structure** preserves the context of the work. It represents the participants, the obligations, the confirmed facts, and the unknown outcomes. Transitions follow explicit rules, and significant changes leave a

verifiable history. The state survives a program restart and the handoff of work to another executor.

4. From the Regulator Model to the Actor Graph

In 1970, Roger Conant and W. Ross Ashby published “Every Good Regulator of a System Must Be a Model of That System.” Its main result ties the quality of regulation to the existence of a model of the regulated system [4].

The theorem concerns the simplest optimal regulator under stated assumptions. It does not prescribe a particular form for the model or a way of storing state.

“Most software today is very much like an Egyptian pyramid . . . just done by brute force and thousands of slaves.”

—Alan Kay, abridged [5]

I use an actor graph to describe, in a single model, both the participants in the work and the rules of their interaction. In my model of active transaction graphs, a person, a program, and an organization are represented as actors with their own state, interfaces, and rules of change. An organization can be expanded into an internal graph while keeping the same form of description [7].

- **Actors and states.** The vertices of an actor graph denote actors, and the active edges denote mediator actors. Together, the participants’ states form the configuration of the system. The transition graph describes changes in these configurations. These are two levels of description of one system: the set of interacting actors, and the possible course of its work.
- **Noise suppression.** In “On the Necessity of Noise Suppression for Minimal Good Regulators,” I show under what conditions a minimal good regulator factorizes through the pragmatic signal space. For the state structure, this sets a guideline: single out the distinctions needed to choose an action, and do not turn noise into extra work [6].
- **Transactional memory and active edges.** By transactional memory I mean the persisted history of interactions, their results, and their accounting consequences. The term has another meaning as well: software transactional memory (STM), a mechanism for synchronizing access to shared memory. A mediator actor stores the state of an interaction and defines the protocol of acknowledgments, retries, deduplication, and compensation. The interaction itself thereby becomes part of an executable and verifiable model.

Hewitt’s actor model and Erlang/OTP provide means for describing and organizing interacting processes [8, 9]; sagas provide a pattern for long-running operations with compensations [10]; and durable execution, as in Temporal, preserves the course of execution after failures [11]. A stateful mediator actor and accounting consequences are mandatory elements of my model of interaction. This makes it possible to verify the rules of interaction together with the result, the execution trace, and the accounting changes.

5. How to Verify a Continuously Running System

An organization keeps working after individual operations complete: new obligations, events, and goals arise. Its executable model must preserve the constraints at every step of this sequence.

Induction makes it possible to prove that an invariant is preserved over any finite number of transitions. Coinduction and bisimulation provide means to analyze infinite behavior and to match the transitions and observations of the model against those of the implementation [12, 13].

When verifying an actor graph, three questions must be kept apart:

1. **Safety: is the invariant “at most one debit per obligation” preserved?** After a timeout, the mediator actor keeps the `outcome-unknown` state and requests reconciliation. If the debit is confirmed, no retry is performed. A `not-found` response does not by itself remove the uncertainty. A retry is admissible only with the original `operation_id` and only while the bank’s deduplication guarantee holds. Without it, the loop continues reconciliation or triggers escalation. Verification must cover all admissible transitions, including failures and retries.
2. **Liveness: will the outcome of the debit eventually be established?** Reconciliation can preserve safety and yet never terminate. Guaranteeing that the outcome is established requires that the bank eventually provide a

reliable answer, that messages be delivered, and that fair scheduling rule out the indefinite postponement of a step that is continuously enabled. Handing the case over for review changes the executor but does not remove the uncertainty about the outcome of the operation.

3. **Conformance to the specification: does the observed behavior of the implementation fall within what the model allows?** The same completed response is not enough: executions with different accounting consequences are not considered equivalent.

6. The Handoff Test

“There is nothing quite so useless as doing with great efficiency something that should not be done at all.”

—Peter Drucker [14]

If after every failure someone has to find out all over again what happened, the system is forcing people to rebuild its memory by hand. In the bank example, this means re-establishing the fate of the debit before the next step can be chosen.

The test is simple: take one unfinished obligation and hand it off to another executor. Can they determine from the saved state what has been confirmed, what remains unknown, and which action is admissible next? A person can close the control loop. But for an organization, this loop must be explicit, and the state of the work must be preserved and passed on when the participants change.

“Work without a control loop and a state structure isn’t worth doing.”

Related Works by the Author

On the Necessity of Noise Suppression for Minimal Good Regulators. Establishes the factorization result used here to distinguish goal-relevant signal from distinctions that consume regulatory capacity without changing admissible action [6].

Actor Graphs: Triple-Identity Accountable Mediation and Coinductive Disclosure. Develops the actor-graph model through triple-identity accountable mediation and coinductive disclosure, providing the formal background for the mediator actors described in this article [7].

References

- [1] Alan J. Perlis, “Epigrams on Programming,” *ACM SIGPLAN Notices*, vol. 17, no. 9, pp. 7–13, 1982. DOI: [10.1145/947955.1083808](https://doi.org/10.1145/947955.1083808).
- [2] Alan J. Perlis, “Epigrams in Programming,” Yale University, 1982. Available: <https://cs.yale.edu/homes/perlis-alan/quotes.html>. Accessed 28 September 2026.
- [3] Alan J. Perlis, “Epigrams on Programming,” archival scan including meta-epigrams, Carnegie Mellon University Libraries. Available: https://iiif.library.cmu.edu/file/Simon_box00075_fld05959_bdl0003_doc0002/Simon_box00075_fld05959_bdl0003_doc0002.pdf. Accessed 28 September 2026.
- [4] Roger C. Conant and W. Ross Ashby, “Every Good Regulator of a System Must Be a Model of That System,” *International Journal of Systems Science*, vol. 1, no. 2, pp. 89–97, 1970. DOI: [10.1080/00207727008920220](https://doi.org/10.1080/00207727008920220).
- [5] Stuart I. Feldman and Alan C. Kay, “A Conversation with Alan Kay,” *ACM Queue*, vol. 2, no. 9, pp. 20–30, 2004. DOI: [10.1145/1039511.1039523](https://doi.org/10.1145/1039511.1039523).
- [6] Alexander Vityaz, *On the Necessity of Noise Suppression for Minimal Good Regulators: Factorization Theorems and a Closure Conjecture*. ResearchGate preprint, January 2026. DOI: [10.13140/RG.2.2.33143.07843](https://doi.org/10.13140/RG.2.2.33143.07843).
- [7] Alexander Vityaz, *Actor Graphs: Triple-Identity Accountable Mediation and Coinductive Disclosure*. Zenodo, 2026. DOI: [10.5281/zenodo.21995981](https://doi.org/10.5281/zenodo.21995981).
- [8] Carl Hewitt, Peter Bishop, and Richard Steiger, “A Universal Modular Actor Formalism for Artificial Intelligence,” in *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*, pp. 235–245, 1973. Available: https://worrydream.com/refs/Hewitt_1973_-_A_Universal_Modular_Actor_Formalism_for_Artificial_Intelligence.pdf. Accessed 28 September 2026.
- [9] Erlang/OTP Team, “OTP Design Principles: Overview,” Erlang/OTP System Documentation. Available: https://www.erlang.org/doc/system/design_principles.html. Accessed 28 September 2026.
- [10] Hector Garcia-Molina and Kenneth Salem, “Sagas,” in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pp. 249–259, 1987. DOI: [10.1145/38713.38742](https://doi.org/10.1145/38713.38742).
- [11] Temporal Technologies, *Durable Execution: A Technical Guide*, 2023. Available: <https://assets.temporal.io/durable-execution.pdf>. Accessed 28 September 2026.
- [12] Leslie Lamport, “Proving the Correctness of Multiprocess Programs,” *IEEE Transactions on Software Engineering*, vol. SE-3, no. 2, pp. 125–143, 1977. DOI: [10.1109/TSE.1977.229904](https://doi.org/10.1109/TSE.1977.229904).
- [13] Davide Sangiorgi, “On the Origins of Bisimulation and Coinduction,” *ACM Transactions on Programming Languages and Systems*, vol. 31, no. 4, article 15, 2009. DOI: [10.1145/1516507.1516510](https://doi.org/10.1145/1516507.1516510).
- [14] Peter F. Drucker, “Managing for Business Effectiveness,” *Harvard Business Review*, vol. 41, no. 3, pp. 53–60, 1963. Available: <https://hbr.org/1963/05/managing-for-business-effectiveness>. Accessed 28 September 2026.